Course Title:

BIT302 Software Engineering

Course Contents

Unit 2: Software Process Model (8 Hrs.)

Software Process Models, Process Activities, Coping with Change, Process Improvement, Agile Software Development

Unit 2: Software Process Model

Unit 2: Software Process Model

- Software Process Models,
- Process Activities,
- Coping with Change,
- Process Improvement,
- Agile Software Development

Unit 2: Software Process Model

1. Introduction
2. Software process
2.1 Software process models
    2.1.1 The waterfall model
    2.1.2 Incremental development
    2.1.3 Integration and configuration
2.2 Process activities
    2.2.1 Software specification
    2.2.2 Software design and implementation
    2.2.3 Software validation
    2.2.4 Software evolution
2.3 Coping with change
    2.3.1 Prototyping
    2.3.2 Incremental delivery
2.4 Process improvement

Unit 2: Software Process Model

3 Agile Software Development

- 3.1 Agile methods
- 3.2 Agile development techniques
  - 3.2.1 User stories
  - 3.2.2 Refactoring
  - 3.2.3 Test-first development
  - 3.2.4 Pair programming
- 3.3 Agile project management
- 3.4 Scaling agile methods
  - 3.4.1 Practical problems with agile methods
  - 3.4.2 Agile and plan-driven methods
  - 3.4.3 Agile methods for large systems
  - 3.4.4 Agile methods across organizations

Unit 2: Software Process Model

Questions:

Details of

- 2.1.1 The waterfall model
- 2.1.2 Incremental development
- 2.1.3 Integration and configuration

- What is Software process ?
- What are different types of Software process Models (Software development methodology)?
- Explain the working principles of Waterfall model and their use of application area? Explain advantages and disadvantages.
- Explain the working principles of Incremental development and their use of application area? Explain advantages and disadvantages.
- Explain the working principles of Integration and configuration and their use of application area? Explain advantages and disadvantages.

Unit 2: Software Process Model

Software process

2.1 Software process models

2.1.1 The waterfall model

2.1.2 Incremental development

2.1.3 Integration and configuration

Unit 2: Software Process Model

Software process

- A software process is a set of related activities that leads to the production of a software system.
- There are many different types of software systems, and there is no universal software engineering method that is applicable to all of them. The process used in different companies depends on the type of software being developed, the requirements of the software, customer, and the skills of the people writing the software.

However, although there are many different software processes, they all must include, in some form, the four fundamental software engineering activities.

1. Software specification: The functionality of the software and constraints on its operation must be defined.
2. Software development: The software to meet the specification must be produced.
3. Software validation: The software must be validated to ensure that it does what the customer wants.
4. Software evolution: The software must evolve to meet changing customer needs.

Unit 2: Software Process Model

Software process:

These activities are complex activities in themselves, and they include sub-activities such as requirements validation, architectural design, and unit testing. Processes also include other activities, such as software configuration management and project planning that support production activities.

When describing processes, it is also important to describe who is involved, what is produced, and conditions that influence the sequence of activities:

1. Products or deliverables are the outcomes of a process activity. For example, the outcome of the activity of architectural design may be a model of the software architecture.
2. Roles reflect the responsibilities of the people involved in the process. Examples of roles are project manager, configuration manager, and programmer.
3. Pre- and post conditions are conditions that must hold before and after a process activity has been enacted or a product produced. For example, before architectural design begins, a precondition may be that the consumer has approved all requirements; after this activity is finished, a post condition might be that the UML models describing the architecture have been reviewed.

Unit 2: Software Process Model

Software process:

A generic process framework for software engineering encompasses five activities:

1. **Communication.** Before any technical work can commence, it is critically important to communicate and collaborate with the customer (and other stakeholders). The intent is to understand stakeholders' objectives for the project and to gather requirements that help define software features and functions.
2. **Planning.** Any complicated journey can be simplified if a map exists. A software project is a complicated journey, and the planning activity creates a "map" that helps guide the team as it makes the journey. The map—called a software project plan—defines the software engineering work by describing the technical tasks to be conducted, the risks that are likely, the resources that will be required, the work products to be produced, and a work schedule.

Unit 2: Software Process Model

Software process:

A generic process framework for software engineering encompasses five activities:

3. **Modeling.** Whether you're a landscaper, a bridge builder, an aeronautical engineer, a carpenter, or an architect, you work with models every day. You create a "sketch" of the thing so that you'll understand the big picture—what it will look like architecturally, how the constituent parts fit together, and many other characteristics. If required, you refine the sketch into greater and greater detail in an effort to better understand the problem and how you're going to solve it. A software engineer does the same thing by creating models to better understand software requirements and the design that will achieve those requirements.
4. **Construction.** This activity combines code generation (either manual or automated) and the testing that is required to uncover errors in the code.
5. **Deployment.** The software (as a complete entity or as a partially completed increment) is delivered to the customer who evaluates the delivered product and provides feedback based on the evaluation.

Unit 2: Software Process Model

2.1 Software process models

2.1.1 The waterfall model

2.1.2 Incremental development

2.1.3 Integration and configuration

Unit 2: Software Process Model

Software process models:

- A software process model is an abstraction of the software development process. The models specify the stages and order of a process.
- So, software process model as a representation of the order of activities of the process and the sequence in which they are performed.

Software Process Model

In many projects, the SDLC phases and steps proceed in a logical path from start to finish.

- In other projects, the project teams move through the steps consecutively(sequentially), incrementally, iteratively, or in other patterns.
- In this section, we describe the phases, the actions, and some of the techniques that are used to accomplish the steps at a very high level.

Unit 2: Software Process Model

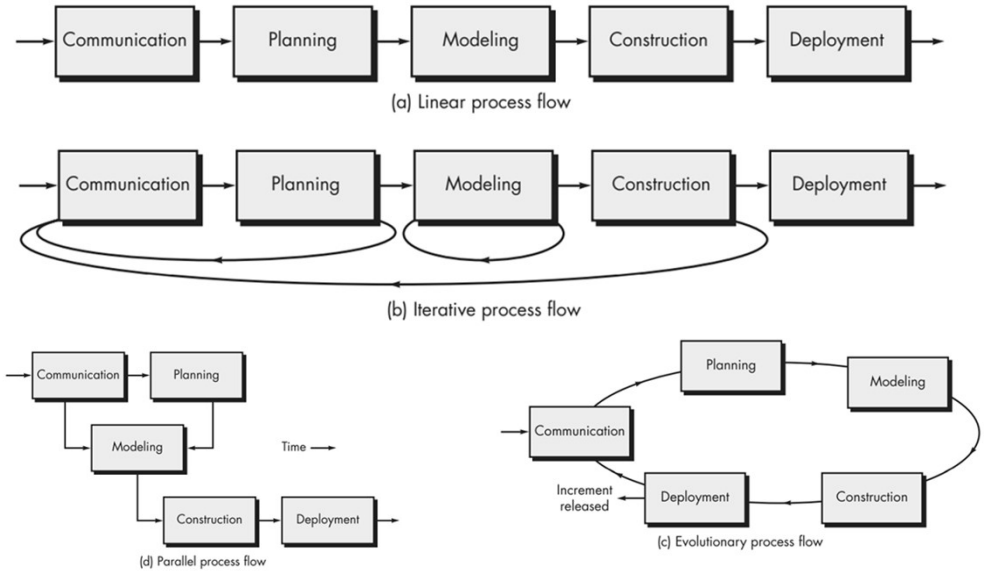
Software process in the aspect of process flow - describes how the framework activities and the actions and tasks that occur within each framework activity are organized with respect to sequence and time.

Software process in the aspect of process flow is categories as

1. **A linear process flow** executes each of the five framework activities in sequence, beginning with communication and ending with deployment.
2. **An iterative process flow** repeats one or more of the activities before proceeding to the next.
3. **An evolutionary process flow** executes the activities in a "circular" manner. Each circuit through the activities leads to a more complete version of the software
4. **A parallel process flow** executes one or more activities in parallel with other activities (e.g., modeling for one aspect of the software might be executed in parallel with construction of another aspect of the software).

Unit 2: Software Process Model

Process Flow:



Unit 2: Software Process Model

Software process models:

Some general process models are

1. **The waterfall model:** This takes the fundamental process activities of specification, development, validation, and evolution and represents them as separate process phases such as requirements specification, software design, implementation, and testing.
2. **Incremental development:** This approach interleaves the activities of specification, development, and validation. The system is developed as a series of versions (increments), with each version adding functionality to the previous version.
3. **Integration and configuration:** This approach relies on the availability of reusable components or systems. The system development process focuses on configuring these components for use in a new setting and integrating them into a system.

Unit 2: Software Process Model

Software process models:

- There is no universal process model that is right for all kinds of software development. The right process depends on the customer and regulatory requirements, the environment where the software will be used, and the type of software being developed.
- For example, safety-critical software is usually developed using a waterfall process as lots of analysis and documentation is required before implementation begins. Software products are now always developed using an incremental process model. Business systems are increasingly being developed by configuring existing systems (off-the-shelf systems) and integrating these to create a new system with the functionality that is required.
- Parts of the system that are well understood can be specified and developed using a waterfall-based process or may be bought in as off-the-shelf systems for configuration. Other parts of the system, which are difficult to specify in advance, should always be developed using an incremental approach. In both cases, software components are likely to be reused.

Unit 2: Software Process Model

Software process models:

- Various attempts have been made to develop "universal" process models that draw on all of these general models. One of the best known of these universal models is the Rational Unified Process (RUP) (Krutchen 2003), which was developed by Rational, a U.S. software engineering company. The RUP is a flexible model that can be instantiated in different ways to create processes that resemble any of the general process models discussed here. The RUP has been adopted by some large software companies (notably IBM), but it has not gained widespread acceptance.

Unit 2: Software Process Model

Software process models:

Details of

2.1.1 The waterfall model

2.1.2 Incremental development

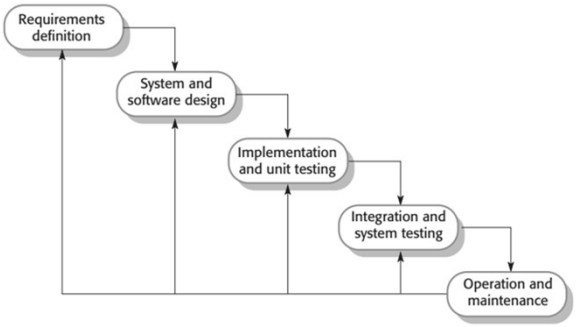
2.1.3 Integration and configuration

- What are different types of Software process Models (Software development methodology)?
- Explain the working principles of Waterfall model and their use of application area? Explain advantages and disadvantages.
- Explain the working principles of Incremental development and their use of application area? Explain advantages and disadvantages.
- Explain the working principles of Integration and configuration and their use of application area? Explain advantages and disadvantages.

Unit 2: Software Process Model

The waterfall model

- The waterfall model is an example of a plan-driven process. In principle at least, you plan and schedule all of the process activities before starting software development.
- It presents the software development process as a number of stages. Because of the cascade from one phase to another, this model is known as the waterfall model or software life cycle.



1. Software specification:
2. Software development:
3. Software validation:
4. Software evolution:

Unit 2: Software Process Model

The waterfall model

The stages of the waterfall model directly reflect the fundamental software development activities:

1. Requirements analysis and definition: The system's services, constraints, and goals are established by consultation with system users. They are then defined in detail and serve as a system specification.
2. System and software design: The systems design process allocates the requirements to either hardware or software systems. It establishes an overall system architecture. Software design involves identifying and describing the fundamental software system abstractions and their relationships.
3. Implementation and unit testing: During this stage, the software design is realized as a set of programs or program units. Unit testing involves verifying that each unit meets its specification.

Unit 2: Software Process Model

The waterfall model

4. Integration and system testing: The individual program units or programs are integrated and tested as a complete system to ensure that the software requirements have been met. After testing, the software system is delivered to the customer.
5. Operation and maintenance: Normally, this is the longest life-cycle phase. The system is installed and put into practical use. Maintenance involves correcting errors that were not discovered in earlier stages of the life cycle, improving the implementation of system units, and enhancing the system's services as new requirements are discovered.

In principle, the result of each phase in the waterfall model is one or more documents that are approved ("signed off"). The following phase should not start until the previous phase has finished. The software process, in practice, is never a simple linear model but involves feedback from one phase to another.

Unit 2: Software Process Model

The waterfall model

In reality, software has to be flexible and accommodate change as it is being developed. The waterfall model is only appropriate for some types of system:

1. Embedded systems where the software has to interface with hardware systems. Because of the inflexibility of hardware, it is not usually possible to delay decisions on the software's functionality until it is being implemented.
2. Critical systems where there is a need for extensive safety and security analysis of the software specification and design. In these systems, the specification and design documents must be complete so that this analysis is possible. Safety related problems in the specification and design are usually very expensive to correct at the implementation stage.
3. Large software systems that are part of broader engineering systems developed by several partner companies. The hardware in the systems may be developed using a similar model, and companies find it easier to use a common model for hardware and software. Furthermore, where several companies are involved, complete specifications may be needed to allow for the independent development of different subsystems.

Unit 2: Software Process Model

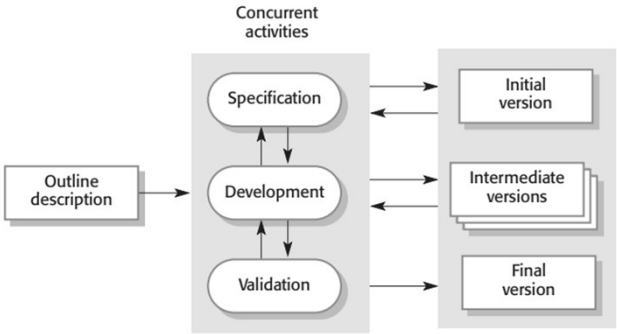
The waterfall model

- The waterfall model is not the right process model in situations where informal team communication is possible and software requirements change quickly. Iterative development and agile methods are better for these systems.
- Because of the high costs of developing a formal specification, this development model is rarely used except for critical systems engineering.

Unit 2: Software Process Model

Incremental development:

- Incremental development is based on the idea of developing an initial implementation, getting feedback from users and others, and evolving the software through several versions until the required system has been developed.
- Specification, development, and validation activities are interleaved rather than separate, with rapid feedback across activities.



Unit 2: Software Process Model

Incremental development:

- Incremental development in some form is now the most common approach for the development of application systems and software products. This approach can be either plan-driven, agile or, more usually, a mixture of these approaches.
- In a plan-driven approach, the system increments are identified in advance; if an agile approach is adopted, the early increments are identified, but the development of later increments depends on progress and customer priorities.
- Incremental software development, which is a fundamental part of agile development methods, is better than a waterfall approach for systems whose requirements are likely to change during the development process.
- By developing the software incrementally, it is cheaper and easier to make changes in the software as it is being developed.

Unit 2: Software Process Model

Incremental development:

- Each increment or version of the system incorporates some of the functionality that is needed by the customer.
- Generally, the early increments of the system include the most important or most urgently required functionality. This means that the customer or user can evaluate the system at a relatively early stage in the development to see if it delivers what is required. If not, then only the current increment has to be changed and, possibly, new functionality defined for later increments.

Unit 2: Software Process Model

Incremental development:

Incremental development has three major advantages over the waterfall model:

1. The cost of implementing requirements changes is reduced. The amount of analysis and documentation that has to be redone is significantly less than is required with the waterfall model.
2. It is easier to get customer feedback on the development work that has been done. Customers can comment on demonstrations of the software and see how much has been implemented. Customers find it difficult to judge progress from software design documents.
3. Early delivery and deployment of useful software to the customer is possible, even if all of the functionality has not been included. Customers are able to use and gain value from the software earlier than is possible with a waterfall process.

Unit 2: Software Process Model

Incremental development:

From a management perspective, the incremental approach has two problems:

1. The process is not visible. Managers need regular deliverables to measure progress. If systems are developed quickly, it is not cost effective to produce documents that reflect every version of the system.
2. System structure tends to degrade as new increments are added. Regular change leads to messy code as new functionality is added in whatever way is possible. It becomes increasingly difficult and costly to add new features to a system. To reduce structural degradation and general code messiness, agile methods suggest that you should regularly refactor (improve and restructure) the software.

Unit 2: Software Process Model

Integration and configuration:

- In the majority of software projects, there is some software reuse. This often happens informally when people working on the project know of or search for code that is similar to what is required. They look for these, modify them as needed, and integrate them with the new code that they have developed.
- This informal reuse takes place regardless of the development process that is used. However, since 2000, software development processes that focus on the reuse of existing software have become widely used.

Unit 2: Software Process Model

Integration and configuration:

Figure shows a general process model for reuse-based development, based on integration and configuration.

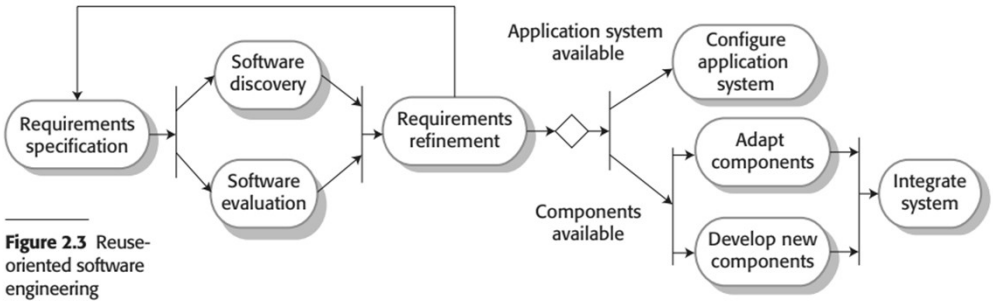


Figure 2.3 Reuse-oriented software engineering

Unit 2: Software Process Model

Integration and configuration:

The stages in this process are:

1. Requirements specification: The initial requirements for the system are proposed. These do not have to be elaborated in detail but should include brief descriptions of essential requirements and desirable system features.

2. Software discovery and evaluation: Given an outline of the software requirements, a search is made for components and systems that provide the functionality required. Candidate components and systems are evaluated to see if they meet the essential requirements and if they are generally suitable for use in the system.

Unit 2: Software Process Model

Integration and configuration:

3. Requirements refinement: During this stage, the requirements are refined using information about the reusable components and applications that have been discovered. The requirements are modified to reflect the available components, and the system specification is re-defined. Where modifications are impossible, the component analysis activity may be reentered to search for alternative solutions.

4. Application system configuration: If an off-the-shelf application system that meets the requirements is available, it may then be configured for use to create the new system.

5. Component adaptation and integration: If there is no off-the-shelf system, individual reusable components may be modified and new components developed. These are then integrated to create the system.

Unit 2: Software Process Model

Integration and configuration:

Advantages:

- Reuse-oriented software engineering, based around configuration and integration, has the obvious advantage of reducing the amount of software to be developed and so reducing cost and risks.
- It usually also leads to faster delivery of the software. However, requirements compromises are inevitable, and this may lead to a system that does not meet the real needs of users.

Disadvantage:

- Furthermore, some control over the system evolution is lost as new versions of the reusable components are not under the control of the organization using them.

Unit 2: Software Process Model

1. Introduction
2. Software process
2.1 Software process models
    2.1.1 The waterfall model
    2.1.2 Incremental development
    2.1.3 Integration and configuration
2.2 Process activities
    2.2.1 Software specification
    2.2.2 Software design and implementation
    2.2.3 Software validation
    2.2.4 Software evolution
2.3 Coping with change
    2.3.1 Prototyping
    2.3.2 Incremental delivery
2.4 Process improvement

Unit 2: Software Process Model

2.2 Process activities

2.2.1 Software specification
2.2.2 Software design and implementation
2.2.3 Software validation
2.2.4 Software evolution

Software processes are the activities involved in producing a software system.

Software Process model as a representation of the order of activities of the process and the sequence in which they are performed.

Unit 2: Software Process Model

Q&A:

- What are the basic software process activities? Explain.
- What are the main key activities in the Software Specification (requirements engineering process).
- What are the key activities of the software development (design process for information systems) ?
- What are the testing process or types of testing in validation phase of software process?
- What is Software Evolution? Explain the evolutionary process.

Unit 2: Software Process Model

Process activities:

- The four basic process activities of specification, development, validation, and evolution are organized differently in different development processes. In the waterfall model, they are organized in sequence, whereas in incremental development they are interleaved.
- How these activities are carried out depends on the type of software being developed, the experience and competence of the developers, and the type of organization developing the software.
- Real software processes are interleaved sequences of technical, collaborative, and managerial activities with the overall goal of specifying, designing, implementing, and testing a software system.
- Generally, processes are now tool-supported. This means that software developers may use a range of software tools to help them, such as requirements management systems, design model editors, program editors, automated testing tools, and debuggers.

Unit 2: Software Process Model

Process activities: The four basic process activities are

1. Software specification
    a. Requirements elicitation and analysis
    b. Requirements specification
    c. Requirements validation
2. Software design and implementation
    a. Architectural design
    b. Database design
    c. Interface design
    d. Component selection and design
3. Software validation
    a. Component testing
    b. System testing
    c. Customer testing
4. Software evolution

Unit 2: Software Process Model

1. Software specification:

There are three main activities in the requirements engineering process:

1. **Requirements elicitation and analysis:** This is the process of deriving the system requirements through observation of existing systems, discussions with potential users and procurers, task analysis, and so on. This may involve the development of one or more system models and prototypes. These help you understand the system to be specified.
2. **Requirements specification:** Requirements specification is the activity of translating the information gathered during requirements analysis into a document that defines a set of requirements. Two types of requirements may be included in this document. User requirements are abstract statements of the system requirements for the customer and end-user of the system; system requirements are a more detailed description of the functionality to be provided.
3. **Requirements validation:** This activity checks the requirements for realism, consistency, and completeness. During this process, errors in the requirements document are inevitably discovered. It must then be modified to correct these problems.

Unit 2: Software Process Model

2. Software design and implementation:

Four activities that may be part of the design process for information systems are:

1. **Architectural design**, where you identify the overall structure of the system, the principal components (sometimes called subsystems or modules), their relationships, and how they are distributed.
2. **Database design**, where you design the system data structures and how these are to be represented in a database. Again, the work here depends on whether an existing database is to be reused or a new database is to be created.
3. **Interface design**, where you define the interfaces between system components. This interface specification must be unambiguous. With a precise interface, a component may be used by other components without them having to know how it is implemented. Once interface specifications are agreed, the components can be separately designed and developed.
4. **Component selection and design**, where you search for reusable components and, if no suitable components are available, design new software components. The design at this stage may be a simple component description with the implementation details left to the programmer. Alternatively, it may be a list of changes to be made to a reusable component or a detailed design model expressed in the UML. The design model may then be used to automatically generate an implementation.

Unit 2: Software Process Model

2. Software design and implementation:

The activities in the design process vary, depending on the type of system being developed. For example, real-time systems require an additional stage of timing design but may not include a database, so there is no database design involved.

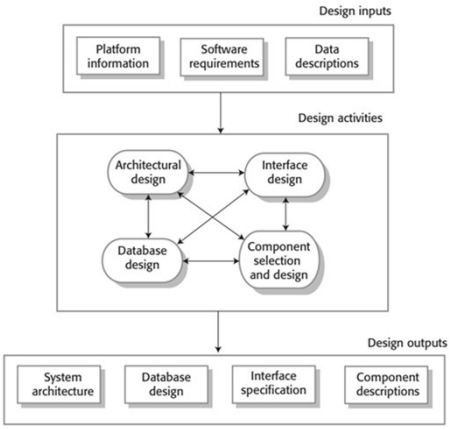


Figure: A general descriptions model of the design process

Unit 2: Software Process Model

3. Software validation:

Software validation or, more generally, verification and validation (V & V) is intended to show that a system both conforms to its specification and meets the expectations of the system customer.
The stages in the testing process are:

Component testing: The components making up the system are tested by the people developing the system. Each component is tested independently, without other system components. Components may be simple entities such as functions or object classes or may be coherent groupings of these entities.

System testing: System components are integrated to create a complete system. This process is concerned with finding errors that result from unanticipated interactions between components and component interface problems. It is also concerned with showing that the system meets its functional and non-functional requirements, and testing the emergent system properties.

Unit 2: Software Process Model

3. Software validation:

Customer testing: This is the final stage in the testing process before the system is accepted for operational use. The system is tested by the system customer (or potential customer) rather than with simulated test data. Customer testing may also reveal requirements problems where the system's facilities do not really meet the users' needs or the system performance is unacceptable. For products, customer testing shows how well the software product meets the customer's needs.

Delivering a system to several prospective users who consent to use it as part of beta testing. They inform the system designers of any issues. This puts the product in actual use and identifies faults that the product creators might not have foreseen. The software product may then be changed and released for additional beta testing or for general sale following this feedback.

Unit 2: Software Process Model

3. Software validation:

Testing phases in a plan-driven software process:

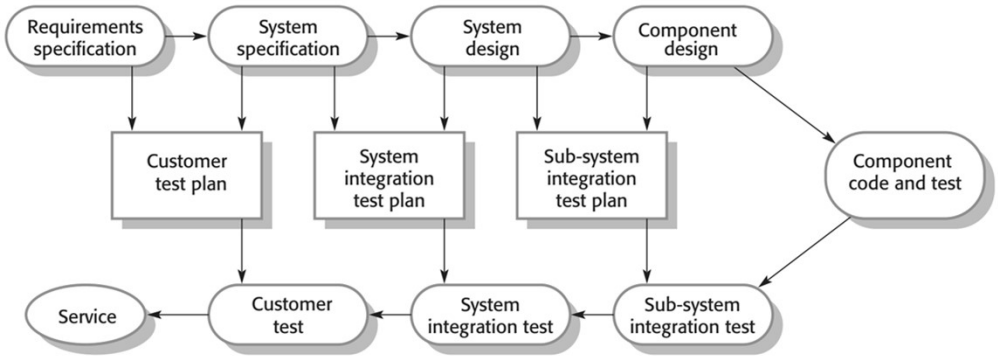
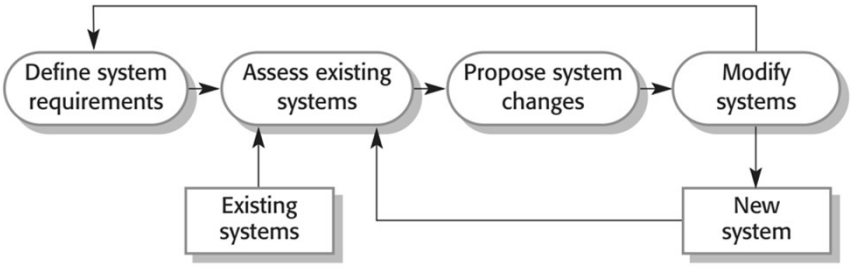


Figure illustrates how test plans are the link between testing and development activities. This is sometimes called the V-model of development (turn it on its side to see the V). The V-model shows the software validation activities that correspond to each stage of the waterfall process model.

Unit 2: Software Process Model

4. Software evolution:

Software engineering as an evolutionary process (Figure below) where software is continually changed over its lifetime in response to changing requirements and customer needs.



The flexibility of software is one of the main reasons why more and more software is being incorporated into large, complex systems while , it is very expensive to make changes to the hardware design.

Unit 2: Software Process Model

2.3 Coping with change

2.3.1 Prototyping

2.3.2 Incremental delivery

- What do you understand by coping with change?
- How to reduce the costs of rework ?
- Explain methods to handle the changes in software?
- What is prototyping? Where and how does it help in software development process?
- What are the advantages of Incremental delivery of software development model? Explain

Unit 2: Software Process Model

Coping with change:

Change is inevitable in all large software projects. The system requirements change as businesses respond to external pressures, competition, and changed management priorities. As new technologies become available, new approaches to design and implementation become possible. Therefore whatever software process model is used, it is essential that it can accommodate changes to the software being developed.

Change adds to the costs of software development because it usually means that work that has been completed has to be redone. This is called rework. It may then be necessary to redesign the system to deliver the new requirements, change any programs that have been developed, and retest the system.

Unit 2: Software Process Model

Coping with change:

Two related approaches may be used to reduce the costs of rework:

1. Change anticipation (Prototyping): where the software process includes activities that can anticipate or predict possible changes before significant rework is required. For example, a prototype system may be developed to show some key features of the system to customers. They can experiment with the prototype and refine their requirements before committing to high software production costs.

2. Change tolerance (Incremental): where the process and software are designed so that changes can be easily made to the system. This normally involves some form of incremental development. Proposed changes may be implemented in increments that have not yet been developed.

Refactoring (improving the structure and organization of a program) is also an important mechanism that supports change tolerance and discuss in Agile methods.

Unit 2: Software Process Model

Coping with change:

So, two ways of coping with change and changing system requirements:

1. System prototyping: where a version of the system or part of the system is developed quickly to check the customer's requirements and the feasibility of design decisions. This is a method of change anticipation as it allows users to experiment with the system before delivery and so refine their requirements. The number of requirements change proposals made after delivery is therefore likely to be reduced.

2. Incremental delivery: where system increments are delivered to the customer for comment and experimentation. This supports both change avoidance and change tolerance. It avoids the premature commitment to requirements for the whole system and allows changes to be incorporated into later increments at relatively low cost.

Unit 2: Software Process Model

Prototyping:

A prototype is an early version of a software system that is used to demonstrate concepts, try out design options, and find out more about the problem and its possible solutions. Rapid, iterative development of the prototype is essential so that costs are controlled and system stakeholders can experiment with the prototype early in the software process.

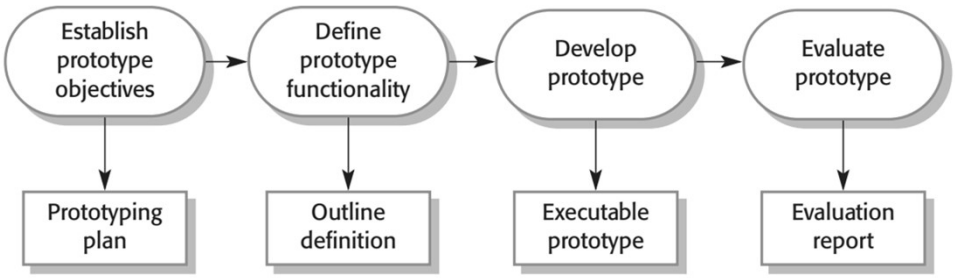
A software prototype can be used in a software development process to help anticipate changes that may be required:

1. In the requirements engineering process, a prototype can help with the elicitation and validation of system requirements.
2. In the system design process, a prototype can be used to explore software solutions and in the development of a user interface for the system

Unit 2: Software Process Model

Prototyping:

A process model for prototype development is shown in Figure. The objectives of prototyping should be made explicit from the start of the process. These may be to develop the user interface, to develop a system to validate functional system requirements, or to develop a system to demonstrate the application to managers. The same prototype usually cannot meet all objectives.



Unit 2: Software Process Model

Incremental delivery:

Incremental delivery is an approach to software development where some of the developed increments are delivered to the customer and deployed for use in their working environment. In an incremental delivery process, customers define which of the services are most important and which are least important to them. A number of delivery increments are then defined, with each increment providing a subset of the system functionality. The allocation of services to increments depends on the service priority, with the highest priority services implemented and delivered first.

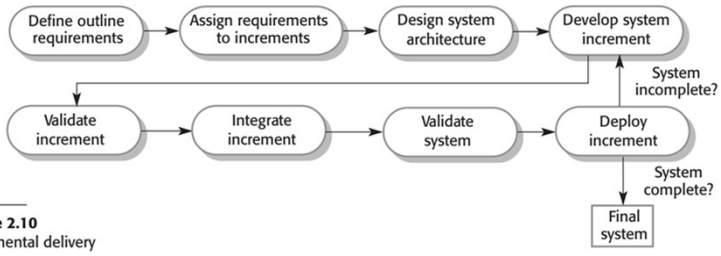


Figure 2.10
Incremental delivery

Unit 2: Software Process Model

Incremental delivery:

Incremental delivery has a number of advantages:

1. Customers can use the early increments as prototypes and gain experience that informs their requirements for later system increments.
2. Customers do not have to wait until the entire system is delivered before they can gain value from it.
3. The process maintains the benefits of incremental development in that it should be relatively easy to incorporate changes into the system.
4. As the highest priority services are delivered first and later increments then integrated, the most important system services receive the most testing. This means that customers are less likely to encounter software failures in the most important parts of the system

Unit 2: Software Process Model

Incremental delivery:

However, there are problems with incremental delivery. In practice, it only works in situations where a brand-new system is being introduced and the system evaluators are given time to experiment with the new system. Key problems with this approach are:

1. Iterative delivery is problematic when the new system is intended to replace an existing system. Users need all of the functionality of the old system and are usually unwilling to experiment with an incomplete new system.
2. Most systems require a set of basic facilities that are used by different parts of the system and can be hard to identify common facilities that are needed by all increments.
3. Specification is developed in conjunction with the software. However, this conflicts with the procurement model of many organizations, where the complete system specification is part of the system development contract.

Unit 2: Software Process Model

2.4 Process improvement

How to handle the demand from industry for cheaper, better software and tighter deadlines?

Unit 2: Software Process Model

Process improvement:

Nowadays, there is a constant demand from industry for cheaper, better software, which has to be delivered to ever-tighter deadlines. Consequently, many software companies have turned to software process improvement as a way of enhancing the quality of their software, reducing costs, or accelerating their development processes.

Two quite different approaches to process improvement and change are used:

1. The process maturity approach, which has focused on improving process and project management and introducing good software engineering practice into an organization.
2. The agile approach, which has focused on iterative development and the reduction of overheads in the software process. The primary characteristics of agile methods are rapid delivery of functionality and responsiveness to changing customer requirements. Agile approaches focus on the code being developed and deliberately minimize formality and documentation.

Unit 2: Software Process Model

Key points:

- Software processes are the activities involved in producing a software system. Software process models are abstract representations of these processes.
- General process models describe the organization of software processes. Examples of these general models include the waterfall model, incremental development, and reusable component configuration and integration.
- Requirements engineering is the process of developing a software specification. Specifications are intended to communicate the system needs of the customer to the system developers.
- Design and implementation processes are concerned with transforming a requirements specification into an executable software system.
- Software validation is the process of checking that the system conforms to its specification and that it meets the real needs of the users of the system.

Unit 2: Software Process Model

Key points:

- Software evolution takes place when you change existing software systems to meet new requirements. Changes are continuous, and the software must evolve to remain useful.
- Processes should include activities to cope with change. This may involve a prototyping phase that helps avoid poor decisions on requirements and design. Processes may be structured for iterative development and delivery so that changes may be made without disrupting the system as a whole.
- Process improvement is the process of improving existing software processes to improve software quality, lower development costs, or reduce development time. It is a cyclic process involving process measurement, analysis, and change.

Unit 2: Software Process Model

3. Agile Software Development

Explain the Agile principles for software development.

What are the key values of Agile Manifesto ?

What are the applicability of Agile method? Also explain the problem with Agile method.

Unit 2: Software Process Model

3 Agile Software Development

- 3.1 Agile methods
- 3.2 Agile development techniques
  - 3.2.1 User stories
  - 3.2.2 Refactoring
  - 3.2.3 Test-first development
  - 3.2.4 Pair programming
- 3.3 Agile project management
- 3.4 Scaling agile methods
  - 3.4.1 Practical problems with agile methods
  - 3.4.2 Agile and plan-driven methods
  - 3.4.3 Agile methods for large systems
  - 3.4.4 Agile methods across organizations

Unit 2: Software Process Model

Agile Software Development:

- Plan-driven software development processes (Like Waterfall) that completely specify the requirements and then design, build, and test a system are not geared to rapid software development. As the requirements change or as requirements problems are discovered, the system design or implementation has to be reworked and retested. As a consequence, a conventional waterfall or specification-based process is usually a lengthy one, and the final software is delivered to the customer long after it was originally specified.
- The need for rapid software development and processes that can handle changing requirements has been recognized for many years. However, faster software development really took off in the late 1990s with the development of the idea of "agile methods" such as Extreme Programming, Scrum, and DSDM (Stapleton 2003).

Unit 2: Software Process Model

Agile Software Development:

- Rapid software development became known as agile development or agile methods. These agile methods are designed to produce useful software quickly.
- Agile methods are incremental development methods in which the increments are small, and, typically, new releases of the system are created and made available to customers every two or three weeks.

Unit 2: Software Process Model

Agile Software Development:

- Rapid development and delivery is now often the most important requirement for software systems. Businesses operate in a fast-changing requirement and it is practically impossible to produce a set of stable software requirements. Software has to evolve quickly to reflect changing business needs.
- Agile software development refers to software development methodologies centered round the idea of iterative development, where requirements and solutions evolve through collaboration between self-organizing cross-functional teams.
- Extreme Programming, Scrum and Kanban are of the most widely used Agile methodologies.

Unit 2: Software Process Model

Agile Software Development:

Agile development methods emerged in the late 1990s whose aim was to radically reduce the delivery time for working software systems.

All of the agile methods that have been proposed share a number of common characteristics:

- Program specification, design, and implementation are interleaved
- The system is developed as a series of frequent versions or increments
- Stakeholders involved in version specification and evaluation
- Extensive tool support (e.g. automated testing tools) used to support development
- Minimal documentation - focus on working code

Unit 2: Software Process Model

The Agile Manifesto consists of four key values:

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Unit 2: Software Process Model

The principles of agile:

There are 12 key principles that guide agile project management.

1. Customer satisfaction is always the highest priority and is achieved through rapid and continuous delivery.
2. Changing environments are included at any stage of the process to provide the customer with a competitive advantage.
3. A product or service is delivered with higher frequency.
4. Stakeholders and developers collaborate closely on a daily basis.
5. All stakeholders and team members remain motivated for optimal project outcomes, while teams are provided with all the necessary tools and support, and are trusted to accomplish project goals.
6. Face-to-face meetings are believed the most efficient and effective format for project success.

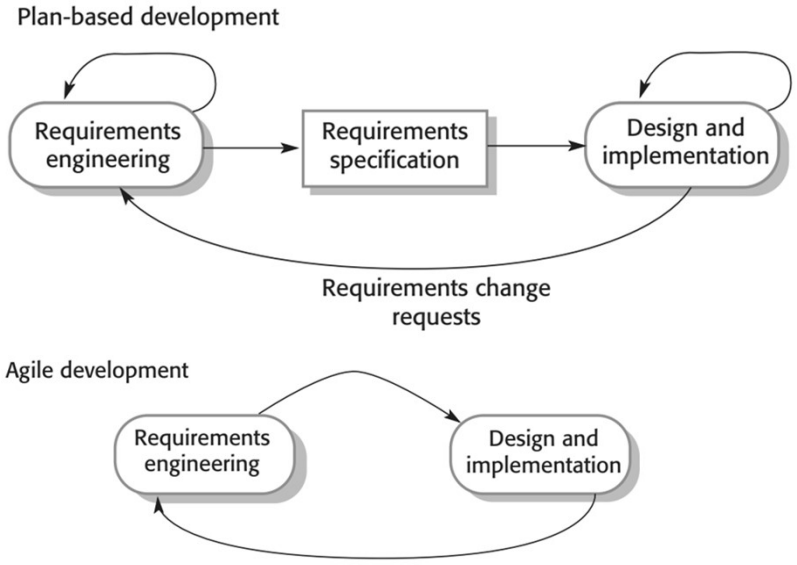
Unit 2: Software Process Model

The principles of agile:

7. A final working product is the ultimate measure of success.
8. Sustainable development is accomplished through agile processes whereby development teams and stakeholders are able to maintain a constant and ongoing pace.
9. Agility is enhanced through a continuous focus on technical excellence and proper design.
10. Simplicity is an essential element.
11. Self-organizing teams are most likely to develop the best architectures and designs and to meet requirements.
12. Regular intervals are used by teams to improve efficiency through fine-tuning behaviors.

Unit 2: Software Process Model

Plan-based vs agile development:



Unit 2: Software Process Model

Plan-based vs agile development:

Note: Requirements document "what is needed - they shouldn't specify the how, but the what". While Specifications document "how to achieve the requirements - they should specify the how".

Plan-driven development

- A plan-driven approach to software engineering is based around separate development stages with the outputs to be produced at each of these stages planned in advance. Not necessarily waterfall model: plan-driven, incremental development is possible. Iteration occurs within activities.

Agile development

- Specification, design, implementation and testing are inter-leaved and the outputs from the development process are decided through a process of negotiation during the software development process.

Most projects include elements of plan-driven and agile processes. Deciding on the balance depends on many technical, human, and organizational issues.

Unit 2: Software Process Model

Agile methods:

The philosophy behind agile methods is reflected in the agile manifesto (http://agilemanifesto.org) issued by the leading developers of these methods. This manifesto states:

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Embrace change	Expect the system requirements to change, and so design the system to accommodate these changes.
Incremental delivery	The software is developed in increments, with the customer specifying the requirements to be included in each increment.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.
People, not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.

Unit 2: Software Process Model

Agile method applicability:

- Product development where a software company is developing a small or medium-sized product.
- Custom system development within an organization, where there is a clear **commitment from the customer** to become involved in the development process and where there are not a lot of external rules and regulations that affect the software.

Because of their focus on small, tightly-integrated teams, there are **problems in scaling** agile methods to large systems.

Unit 2: Software Process Model

Problems with agile methods:

- It can be difficult to keep the interest of **customers** who are involved in the process.
- **Team members** may be unsuited to the intense involvement that characterizes agile methods.
- Prioritizing changes can be difficult where there are **multiple stakeholders**.
- Maintaining simplicity requires extra work.
- **Contracts** may be a problem as with other approaches to iterative development.

Agile Software Development

Extreme Programming:

Extreme Programming (XP) process/Techniques:

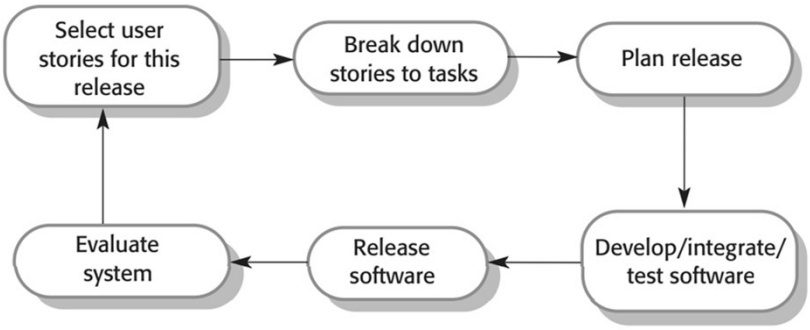
- Software engineer Ken Beck introduced XP in the 90s with the goal of finding ways to write high-qualitative software quickly and being able to adapt to customers' changing requirements. In 1999, he refined XP approaches in the book Extreme Programming. XP is a set of engineering practices.
- Developers have to go beyond their capabilities while performing these practices. That's where the "extreme" in the framework's title comes from.

Agile Software Development

Extreme Programming (XP) process/Techniques:

In XP, requirements are expressed as scenarios (called user stories), which are implemented directly as a series of tasks. Programmers work in pairs and develop tests for each task before writing the code. All tests must be successfully executed when new code is integrated into the system. There is a short time gap between releases of the system.

Figure: The XP release cycle:



Agile Software Development

Extreme Programming (XP) practices/principles:

- Collective ownership
- Continuous integration
- Incremental planning
- On-site customer
- Pair programming
- Refactoring
- Simple design
- Small release
- Sustainable pace
- Test first development

Principle or practice	Description
Collective ownership	The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers take responsibility for all of the code. Anyone can change anything.
Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.
Incremental planning	Requirements are recorded on "story cards," and the stories to be included in a release are determined by the time available and their relative priority. The developers break these stories into development "tasks." See Figures 3.5 and 3.6.
On-site customer	A representative of the end-user of the system (the Customer) should be available full time for the use of the XP team. In an extreme programming process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.
Pair programming	Developers work in pairs, checking each other's work and providing the support to always do a good job.
Refactoring	All developers are expected to refactor the code continuously as soon as potential code improvements are found. This keeps the code simple and maintainable.
Simple design	Enough design is carried out to meet the current requirements and no more.
Small releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.
Sustainable pace	Large amounts of overtime are not considered acceptable, as the net effect is often to reduce code quality and medium-term productivity.
Test first development	An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.

Unit 2: Software Process Model

3.2 Agile development techniques

3.2.1 User stories

3.2.2 Refactoring

3.2.3 Test-first development

3.2.4 Pair programming

Unit 2: Software Process Model

3.2.1 User stories

- Agile method do not have a separate requirements engineering activity. Rather, Agile method integrate requirements elicitation with development.
- To make this easier, the idea of "user stories" was developed where a user story is a scenario of use that might be experienced by a system user.
- The system customer works closely with the development team and discusses these scenarios with other team members.
- Together, they develop a "story card" that briefly describes a story that encapsulates the customer needs. The development team then aims to implement that scenario in a future release of the software.

Unit 2: Software Process Model

3.2.1 User stories

- Once the story cards have been developed, the development team breaks these down into tasks and estimates the effort and resources required for implementing each task. This usually involves discussions with the customer to refine the requirements. The customer then prioritizes the stories for implementation, choosing those stories that can be used immediately to deliver useful business support.
- The intention is to identify useful functionality that can be implemented in about two weeks, when the next release of the system is made available to the customer.
- User stories can be helpful in getting users involved in suggesting requirements during an initial pre-development requirements elicitation activity.

Unit 2: Software Process Model

XP Development process:

- In XP, a customer or user is part of the XP team and is responsible for making decisions on requirements.
- User requirements are expressed as scenarios or user stories.
- These are written on cards and the development team break them down into implementation tasks. These tasks are the basis of schedule and cost estimates.
- The customer chooses the stories for inclusion in the next release based on their priorities and the schedule estimates.

Unit 2: Software Process Model

User stories: Example -A 'prescribing medication' story

Prescribing medication

Kate is a doctor who wishes to prescribe medication for a patient attending a clinic. The patient record is already displayed on her computer so she clicks on the medication field and can select 'current medication', 'new medication' or 'formulary'.

If she selects 'current medication', the system asks her to check the dose; If she wants to change the dose, she enters the new dose then confirms the prescription.

If she chooses 'new medication', the system assumes that she knows which medication to prescribe. She types the first few letters of the drug name. The system displays a list of possible drugs starting with these letters. She chooses the required medication and the system responds by asking her to check that the medication selected is correct. She enters the dose then confirms the prescription.

If she chooses 'formulary', the system displays a search box for the approved formulary. She can then search for the drug required. She selects a drug and is asked to check that the medication is correct. She enters the dose then confirms the prescription.

The system always checks that the dose is within the approved range. If it isn't, Kate is asked to change the dose.

After Kate has confirmed the prescription, it will be displayed for checking. She either clicks 'OK' or 'Change'. If she clicks 'OK', the prescription is recorded on the audit database. If she clicks on 'Change', she reenters the 'Prescribing medication' process.

Unit 2: Software Process Model

User stories: Examples of task cards for prescribing medication

Task 1: Change dose of prescribed drug

Task 2: Formulary selection

Task 3: Dose checking

Dose checking is a safety precaution to check that the doctor has not prescribed a dangerously small or large dose.

Using the formulary id for the generic drug name, look up the formulary and retrieve the recommended maximum and minimum dose.

Check the prescribed dose against the minimum and maximum. If outside the range, issue an error message saying that the dose is too high or too low. If within the range, enable the 'Confirm' button.

Unit 2: Software Process Model

3.2.2 Refactoring

- Changes will always have to be made to the code being developed. To make these changes easier, the developers of XP suggested that the code being developed should be constantly refactored.
- Refactoring means that the programming team look for possible improvements to the software and implements them immediately. When team members see code that can be improved, they make these improvements even in situations where there is no immediate need for them.
- Refactoring improves the software structure and readability and so avoids the structural deterioration that naturally occurs when software is changed. Examples of refactoring include the reorganization of a class hierarchy to remove duplicate code, the tidying up and renaming of attributes and methods, and the replacement of similar code sections, with calls to methods defined in a program library.

Unit 2: Software Process Model

Refactoring

- Programming team look for possible software improvements and make these improvements even where there is no immediate need for them
- This improves the understandability of the software and so reduces the need for documentation
- Changes are easier to make because the code is well-structured and clear
- However, some changes requires architecture refactoring and this is much more expensive.

Unit 2: Software Process Model

Examples of refactoring

- Re-organization of a class hierarchy to remove duplicate code
- Tidying up and renaming attributes and methods to make them easier to understand
- The replacement of inline code with calls to methods that have been included in a program library

Unit 2: Software Process Model

Refactoring (XP and Change)

- The conventional wisdom in software engineering emphasizes the importance of designing software systems with the expectation of future changes. This approach suggests that investing time and effort upfront to anticipate potential changes can help reduce costs and challenges later in the software's life cycle. However, Extreme Programming (XP), a software development methodology, challenges this conventional wisdom by asserting that it is not feasible to reliably predict and anticipate all future changes.
- XP advocates for a different approach, suggesting that the focus should be on constant code improvement through a practice known as refactoring. Refactoring involves making incremental changes to the codebase to improve its structure, maintainability, and clarity without altering its external behavior. By consistently refining the codebase, XP aims to make it easier to accommodate changes when they become necessary, without attempting to predict the specific changes in advance.

Unit 2: Software Process Model

Refactoring (XP and Change)

- In essence, while the conventional wisdom encourages designing for anticipated changes, XP advocates for a more adaptive and responsive approach through continuous code improvement. This way, software remains flexible and adaptable to changes that may emerge over time, without the need for extensive upfront design based on potentially uncertain future requirements.

Unit 2: Software Process Model

Refactoring (XP and Change)

In Brief,

- Conventional wisdom in software engineering is to design for change. It is worth spending time and effort anticipating changes as this reduces costs later in the life cycle.
- XP, however, maintains that this is not worthwhile as changes cannot be reliably anticipated
- Rather, it proposes constant code improvement (refactoring) to make changes easier when they have to be implemented

Unit 2: Software Process Model

Test-first development (Testing in XP)

- Testing is central to XP and XP has developed an approach where the program is tested after every change has been made.
- XP testing features:
  - Test-first development
  - Incremental test development from scenarios
  - User involvement in test development and validation
  - Automated test harnesses are used to run all component tests each time that a new release is built

Extreme Programming developed a new approach to program testing to address the difficulties of testing without a specification. Testing is automated and is central to the development process, and development cannot proceed until all tests have been successfully executed.

Unit 2: Software Process Model

Test-first development

- XP's test-first philosophy has now evolved into more general test-driven development techniques.
- Test-driven development is one of the most important innovations in software engineering.
- Instead of writing code and then writing tests for that code, you write the tests before you write the code.
- This means that you can run the test as the code is being written and discover problems during development.
- In test-first development, the task implementers have to thoroughly understand the specification so that they can write tests for the system. This means that ambiguities and omissions in the specification have to be clarified before implementation begins.

Unit 2: Software Process Model

Test-first development

- Writing tests before code clarifies the requirements to be implemented.
- Tests are written as programs rather than data so that they can be executed automatically. The test includes a check that it has executed correctly.
  - Usually relies on a testing framework such as Junit. (JUnit is a Regression Testing Framework used by developers to implement unit testing in Java, and accelerate programming speed and increase the quality of code)
- All previous and new tests are run automatically when new functionality is added, thus checking that the new functionality has not introduced errors

Junit: https://www.vogella.com/tutorials/JUnit/article.html

Unit 2: Software Process Model

Test-first development (Customer involvement)

- The role of the customer in the testing process is to help develop acceptance tests for the stories that are to be implemented in the next release of the system.
- The customer who is part of the team writes tests as development proceeds. All new code is therefore validated to ensure that it is what the customer needs.
- However, people adopting the customer role have limited time available and so cannot work full-time with the development team. They may feel that providing the requirements was enough of a contribution and so may be reluctant to get involved in the testing process.

Unit 2: Software Process Model

Test-first development (XP testing difficulties)

- Programmers prefer programming to testing and sometimes they take short cuts when writing tests. For example, they may write incomplete tests that do not check for all possible exceptions that may occur.
- Some tests can be difficult to write incrementally. For example, in a complex user interface, it is often difficult to write unit tests for the code that implements the ‘display logic’ and workflow between screens.
- It difficult to judge the completeness of a set of tests. Although you may have a lot of system tests, your test set may not provide complete coverage.

Unit 2: Software Process Model

3.2.4 Pair Programming

- In XP, programmers work in pairs, sitting together to develop code.
- This helps develop common ownership of code and spreads knowledge across the team.
- It serves as an informal review process as each line of code is looked at by more than 1 person.
- It encourages refactoring as the whole team can benefit from this.
- Measurements suggest that development productivity with pair programming is similar to that of two people working independently.

Unit 2: Software Process Model

Advantages of pair programming:

- It supports the idea of collective ownership and responsibility for the system
  - Individuals are not held responsible for problems with the code. Instead, the team has collective responsibility for resolving these problems.
- It acts as an informal review process because each line of code is looked at by at least two people
- It helps support refactoring, which is a process of software improvement
  - Where pair programming and collective ownership are used, others benefit immediately from the refactoring so they are likely to support the process.

Agile Software Development

Agile project management:

- In any software business, managers need to know what is going on and whether or not a project is likely to meet its objectives and deliver the software on time with the proposed budget. Plan-driven approaches to software development evolved to meet this need. Managers draw up a plan for the project showing what should be delivered, when it should be delivered, and who will work on the development of the project deliverables. A plan-based approach requires a manager to have a stable view of everything that has to be developed and the development processes.
- The **informal planning** and **project control** of agile methods clashed with this business requirement for visibility. Teams were self-organizing, did not produce documentation, and planned development in very short cycles.
- While this can and does work for small companies developing software products, it is inappropriate for larger companies who need to know what is going on in their organization.

Unit 2: Software Process Model

Agile project management:

Like every other professional software development process, agile development has to be managed so that the best use is made of the time and resources available to the team. To address this issue, the Scrum agile method was developed to provide a framework for organizing agile projects and, to some extent at least, provide external visibility of what is going on.

The developers of Scrum wished to make clear that Scrum was not a method for project management in the conventional sense, so they deliberately invented new terminology, such as ScrumMaster, which replaced names such as project manager.

Agile Software Development

Agile project management: Table summarizes Scrum terminology and what it means.

Scrum Team:

- Development team
- Potentially shippable product increment
- Product backlog
- Product owner
- Scrum
- ScrumMaster
- Sprint
- Velocity

Scrum term	Definition
Development team	A self-organizing group of software developers, which should be no more than seven people. They are responsible for developing the software and other essential project documents.
Potentially shippable product increment	The software increment that is delivered from a sprint. The idea is that this should be "potentially shippable," which means that it is in a finished state and no further work, such as testing, is needed to incorporate it into the final product. In practice, this is not always achievable.
Product backlog	This is a list of "to do" items that the Scrum team must tackle. They may be feature definitions for the software, software requirements, user stories, or descriptions of supplementary tasks that are needed, such as architecture definition or user documentation.
Product owner	An individual (or possibly a small group) whose job is to identify product features or requirements, prioritize these for development, and continuously review the product backlog to ensure that the project continues to meet critical business needs. The Product Owner can be a customer but might also be a product manager in a software company or other stakeholder representative.
Scrum	A daily meeting of the Scrum team that reviews progress and prioritizes work to be done that day. Ideally, this should be a short face-to-face meeting that includes the whole team.
ScrumMaster	The ScrumMaster is responsible for ensuring that the Scrum process is followed and guides the team in the effective use of Scrum. He or she is responsible for interfacing with the rest of the company and for ensuring that the Scrum team is not diverted by outside interference. The Scrum developers are adamant that the ScrumMaster should not be thought of as a project manager. Others, however, may not always find it easy to see the difference.
Sprint	A development iteration. Sprints are usually 2 to 4 weeks long.
Velocity	An estimate of how much product backlog effort a team can cover in a single sprint. Understanding a team's velocity helps them estimate what can be covered in a sprint and provides a basis for measuring improving performance.

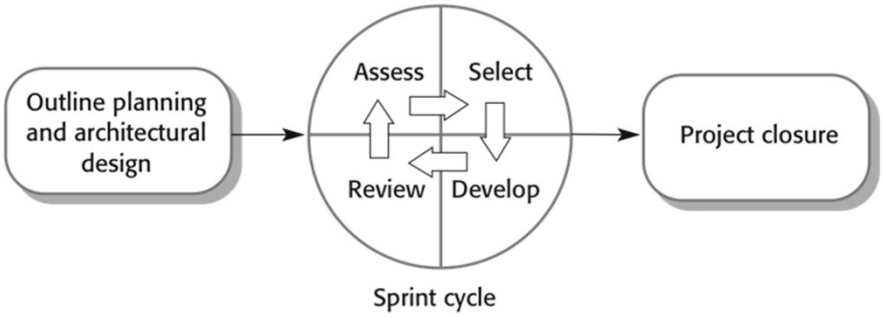
Agile Software Development

Scrum:

- The Scrum approach is a general agile method but its focus is on managing iterative development rather than specific agile practices
- There are three phases in Scrum:
  - The initial phase is an outline planning phase where you establish the general objectives for the project and design the software architecture
  - This is followed by a series of sprint cycles, where each cycle develops an increment of the system
  - The project closure phase wraps up the project, completes required documentation such as system help frames and user manuals and assesses the lessons learned from the project.

Unit 2: Software Process Model

The Scrum process:



Unit 03: Agile Software Development

The Sprint cycle:

- Sprints are fixed length, normally 2–4 weeks. They correspond to the development of a release of the system in XP.
- The starting point for planning is the product backlog, which is the list of work to be done on the project.
- The selection phase involves all of the project team who work with the customer to select the features and functionality to be developed during the sprint.
- Once these are agreed, the team organize themselves to develop the software. During this stage the team is isolated from the customer and the organization, with all communications channelled through the so-called Scrum master
- The role of the Scrum master is to protect the development team from external distractions
- At the end of the sprint, the work done is reviewed and presented to stakeholders. The next sprint cycle then begins.

Unit 03: Agile Software Development

Teamwork in Scrum:

- The 'Scrum master' is a facilitator who arranges daily meetings, tracks the backlog of work to be done, records decisions, measures progress against the backlog and communicates with customers and management outside of the team.
- The whole team attends short daily meetings where all team members share information, describe their progress since the last meeting, problems that have arisen and what is planned for the following day.
- This means that everyone on the team knows what is going on and, if problems arise, can re-plan short-term work to cope with them.

Unit 03: Agile Software Development

Scrum benefits:

- The product is broken down into a set of manageable and understandable chunks
- Unstable requirements do not hold up progress
- The whole team have visibility of everything and consequently team communication is improved
- Customers see on-time delivery of increments and gain feedback on how the product works
- Trust between customers and developers is established and a positive culture is created in which everyone expects the project to succeed

Unit 2: Software Process Model

3.4 Scaling agile methods

3.4.1 Practical problems with agile methods

3.4.2 Agile and plan-driven methods

3.4.3 Agile methods for large systems

3.4.4 Agile methods across organizations

Unit 2: Software Process Model

Scaling agile methods

- Agile methods have proved to be successful for small and medium sized projects that can be developed by a small co-located team
- It is sometimes argued that the success of these methods comes because of improved communications which is possible when everyone is working together
- Scaling up agile methods involves changing these to cope with larger, longer projects where there are multiple development teams, perhaps working in different locations.

Unit 2: Software Process Model

Scaling agile methods (Large systems development)

What is large System?

- Large systems are usually collections of separate, communicating systems, where separate teams develop each system. Frequently, these teams are working in different places, sometimes in different time zones.
- Large systems are 'brownfield systems'(problem spaces needing the development and deployment of new software systems in the immediate presence of existing (legacy) software applications/systems.) , that is they include and interact with a number of existing systems. Many of the system requirements are concerned with this interaction and so don't really lend themselves to flexibility and incremental development.
- Where several systems are integrated to create a system, a significant fraction of the development is concerned with system configuration rather than original code development.

Unit 2: Software Process Model

Scaling agile methods (Large systems development)

Practical problems with agile methods

- Large systems and their development processes are often constrained by external rules and regulations limiting the way that they can be developed.
- Large systems have a long procurement and development time. It is difficult to maintain coherent teams who know about the system over that period as, inevitably, people move on to other jobs and projects.
- Large systems usually have a diverse set of stakeholders. It is practically impossible to involve all of these different stakeholders in the development process.

Three types of problems can arise:

1. lack of product documentation
2. keeping customers involved
3. development team continuity

Unit 2: Software Process Model

Scaling agile methods (Scaling out and scaling up)

- 'Scaling up' is concerned with using agile methods for developing large software systems that cannot be developed by a small team
- 'Scaling out' is concerned with **how** agile methods can be introduced across a large organization with many years of software development experience
- When scaling agile methods, it is essential to maintain agile fundamentals
  - Flexible planning, frequent system releases, continuous integration, test-driven development and good team communications.

Unit 2: Software Process Model

Scaling agile methods (Scaling up to large systems)

- For large systems development, it is not possible to focus only on the code of the system. You need to do more up-front design and system documentation.
- Cross-team communication mechanisms have to be designed and used. This should involve regular phone and video conferences between team members and frequent, short electronic meetings where teams update each other on progress.
- Continuous integration (where the whole system is built every time any developer checks in a change) is practically impossible. However, it is essential to maintain frequent system builds and regular releases of the system.

Unit 2: Software Process Model

Scaling agile methods (Scaling out to large companies)

Agile methods across organizations: It can be difficult to introduce agile methods into large companies for a number of reasons:

- Project managers who do not have experience of agile methods may be reluctant to accept the risk of a new approach.
- Large organizations often have quality procedures and standards that all projects are expected to follow and, because of their bureaucratic nature, these are likely to be incompatible with agile methods.
- Agile methods seem to work best when team members have a relatively high skill level. However, within large organizations, there are likely to be a wide range of skills and abilities.
- There may be cultural resistance to agile methods, especially in those organizations that have a long history of using conventional systems (Agile methods) engineering processes. (Changing from XP to scrum, plan-driven development of large system).

Unit 2: Software Process Model

Agile and plan-driven methods

- A fundamental requirement of scaling agile methods is to integrate them with plan driven approaches.
- Small startup companies can work with informal and short-term planning, but larger companies have to have longer-term plans and budgets for investment, staffing, and business development. Their software development must support these plans, so longer-term software planning is essential.
- Most large “agile” software development projects combine practices from plan-driven and agile approaches. Some are mostly agile, and others are mostly plan-driven but with some agile practices.

Unit 2: Software Process Model

Agile and plan-driven methods:

Factors influencing the choice

of

plan-based or agile development

Unit 2: Software Process Model

Agile and plan-driven methods:

Agile methods were developed and refined in projects to develop small to medium sized business systems and software products, where the software developer controls the specification of the system. Other types of system have attributes such as size, complexity, real-time response, and external regulation that mean a “pure” agile approach is unlikely to work. There needs to be some up-front planning, design, and documentation in the systems engineering process.

Factors influencing the choice of plan-based or agile development are as follows:

Unit 2: Software Process Model

Agile and plan-driven methods: System

1. How large is the system that is being developed? Agile methods are most effective when the system can be developed with a relatively small co-located team who can communicate informally. This may not be possible for large systems that require larger development teams, so a plan-driven approach may have to be used.
2. What type of system is being developed? Systems that require a lot of analysis before implementation (e.g., real-time system with complex timing requirements) usually need a fairly detailed design to carry out this analysis. A plan-driven approach may be best in those circumstances.
3. What is the expected system lifetime? Long-lifetime systems may require more design documentation to communicate the original intentions of the system developers to the support team. However, supporters of agile methods rightly argue that documentation is frequently not kept up to date and is not of much use for long-term system maintenance.
4. Is the system subject to external regulation? If a system has to be approved by an external regulator (e.g., the Federal Aviation Administration approves software that is critical to the operation of an aircraft), then you will probably be required to produce detailed documentation as part of the system safety case.

Unit 2: Software Process Model

Agile and plan-driven methods: Team

1. How good are the designers and programmers in the development team? It is sometimes argued that agile methods require higher skill levels than plan-based approaches in which programmers simply translate a detailed design into code. If you have a team with relatively low skill levels, you may need to use the best people to develop the design, with others responsible for programming.
2. How is the development team organized? If the development team is distributed or if part of the development is being outsourced, then you may need to develop design documents to communicate across the development teams.
3. What technologies are available to support system development? Agile methods often rely on good tools to keep track of an evolving design. If you are developing a system using an IDE that does not have good tools for program visualization and analysis, then more design documentation may be required.

Unit 2: Software Process Model

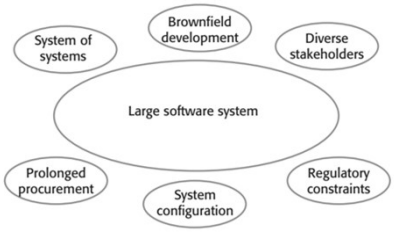
Agile and plan-driven methods: Organization

1. Is it important to have a very detailed specification and design before moving to implementation, perhaps for contractual reasons? If so, you probably need to use a plan-driven approach for requirements engineering but may use agile development practices during system implementation.
2. Is an incremental delivery strategy, where you deliver the software to customers or other system stakeholders and get rapid feedback from them, realistic? Will customer representatives be available, and are they willing to participate in the development team?
3. Are there cultural issues that may affect system development? Traditional engineering organizations have a culture of plan-based development, as this is the norm in engineering. This usually requires extensive design documentation rather than the informal knowledge used in agile processes.

Unit 2: Software Process Model

Large project characteristics

(Six principal factors)

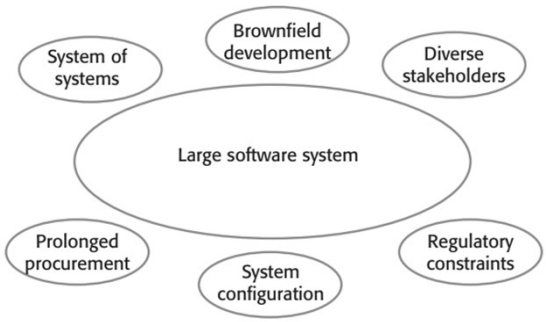


Unit 2: Software Process Model

Large project characteristics:

Agile methods have to evolve to be used for large-scale software development. The fundamental reason for this is that large-scale software systems are much more complex and difficult to understand and manage than small-scale systems or software products.

Six principal factors contribute to this complexity (Large project characteristics):



Unit 2: Software Process Model

Large project characteristics:

Large systems are usually systems of systems - collections of separate, communicating systems, where separate teams develop each system. Frequently, these teams are working in different places, sometimes in different time zones. It is practically impossible for each team to have a view of the whole system. Consequently, their priorities are usually to complete their part of the system without regard for wider systems issues.

Large systems are brownfield systems - that is, they include and interact with a number of existing systems. Many of the system requirements are concerned with this interaction and so don't really lend themselves to flexibility and incremental development. Political issues can also be significant here -often the easiest solution to a problem is to change an existing system. However, this requires negotiation with the managers of that system to convince them that the changes can be implemented without risk to the system's operation.

Unit 2: Software Process Model

Large project characteristics:

System configuration - where several systems are integrated to create a system, a significant fraction of the development is concerned with system configuration rather than original code development. This is not necessarily compatible with incremental development and frequent system integration

Regularity constraints - Large systems and their development processes are often constrained by external rules and regulations limiting the way that they can be developed, that require certain types of system documentation to be produced, and so on. Customers may have specific compliance requirements that may have to be followed, and these may require process documentation to be completed.

Prolonged procedure - Large systems have a long procurement and development time. It is difficult to maintain coherent teams who know about the system over that period as, inevitably, people move on to other jobs and projects.

Unit 2: Software Process Model

Large project characteristics:

Diverse stakeholders - Large systems usually have a diverse set of stakeholders with different perspectives and objectives. For example, nurses and administrators may be the end-users of a medical system, but senior medical staff, hospital managers, and others, are also stakeholders in the system. It is practically impossible to involve all of these different stakeholders in the development process.

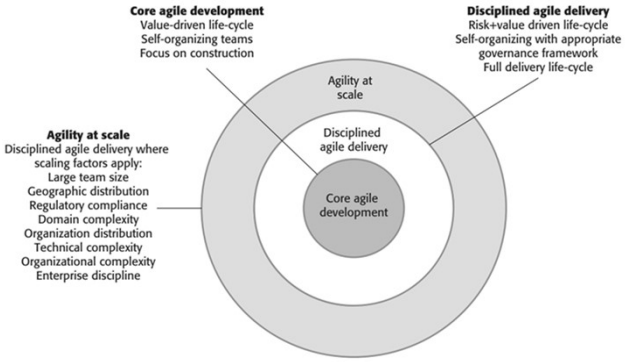
Unit 2: Software Process Model

Agile methods for large systems

Unit 2: Software Process Model

Agile methods for large systems:

Dean Leffingwell, who has a great deal of experience in scaling agile methods, has developed the Scaled Agile Framework (Leffingwell 2007, 2011) to support large-scale, multi-team software development. IBM has also developed a framework for the large-scale use of agile methods called the Agile Scaling Model (ASM). Figure shows an overview of this model.



Unit 2: Software Process Model

Agile methods for large systems:

Scrum has been adapted for large-scale development with the Scrum team model, but multiple Scrum teams are set up. The key characteristics of multi-team Scrum are:

1. Role replication: Each team has a Product Owner for its work component and ScrumMaster. There may be a chief Product Owner and ScrumMaster for the entire project.

2. Product architects: Each team chooses a product architect, and these architects collaborate to design and evolve the overall system architecture.

3. Release alignment: The dates of product releases from each team are aligned so that a demonstrable and complete system is produced.

4. Scrum of Scrums: There is a daily Scrum of Scrums where representatives from each team meet to discuss progress, identify problems, and plan the work to be done that day. Individual team Scrums may be staggered in time so that representatives from other teams can attend if necessary.

Unit 2: Software Process Model

Key points

- Agile methods are incremental development methods that focus on rapid development, frequent releases of the software, reducing process overheads and producing high-quality code. They involve the customer directly in the development process.
- The decision on whether to use an agile or a plan-driven approach to development should depend on the type of software being developed, the capabilities of the development team, and the culture of the company developing the system.
- Extreme programming is a well-known agile method that integrates a range of good programming practices such as frequent releases of the software, continuous software improvement and customer participation in the development team.

Unit 2: Software Process Model

Key points

- A particular strength of extreme programming is the development of automated tests before a program feature is created. All tests must successfully execute when an increment is integrated into a system.
- The Scrum method is an agile method that provides a project management framework. It is centred round a set of sprints, which are fixed time periods when a system increment is developed.
- Scaling agile methods for large systems is difficult. Large systems need up-front design and a lot of documentation.

Unit-1: Introduction – Q&A

1. **Four fundamental software engineering activities**
   a. Software specification
   b. Software development
   c. Software validation
   d. Software evolution
2. **Five fundamental activities of software engineering.**
   a. Communication
   b. Planning
   c. Modeling
   d. Construction
   e. Deployment
3. **Software process models: (Some general process models are)**
   a. The waterfall model
   b. Incremental development
   c. Integration and configuration
4. **Waterfall model -Fundamental software development activities of Waterfall model.**
   a. Requirements analysis and definition
   b. System and software design
   c. Implementation and unit testing
   d. Integration and system testing
   e. Operation and maintenance

Unit-1: Introduction – Q&A

5. **The waterfall model is suitable for what types of system types?**
    a. Embedded systems
    b. Critical systems
    c. Large software systems
6. **What is Incremental development?**

➢ Incremental development is based on the idea of developing an initial implementation, getting feedback from users and others, and evolving the software through several versions until the required system has been developed.

5. **Three major advantages of Incremental development over the waterfall model.**
    a. cost of implementing requirements changes is reduced
    b. easier to get customer feedback on the development work
    c. Early delivery and deployment of useful software to the customer is possible
6. **The incremental approach has two problems**
    a. The process is not visible. Managers need regular deliverables to measure progress. it is not cost effective to produce documents
    b. System structure tends to degrade as new increments are added
7. **The stages in the process of Integration and configuration are**
    a. Requirements specification:
    b. Software discovery and evaluation
    c. Requirements refinement
    d. Application system configuration
    e. Component adaptation and integration

Unit-1: Introduction – Q&A

5. **Software requirement specification: What are three main activities in the requirements engineering process?**
    a. Requirements elicitation and analysis
    b. Requirements specification
    c. Requirements validation
6. **Software design and implementation: Four activities that may be part of the design process for information systems are:**
    a. Architectural design
    b. Database design
    c. Interface design
    d. Component selection and design
    6. **Software validation: The stages in the testing process are:**
    a. Component testing
    b. System testing
    c. Customer testing
7. **Coping with change: Two related approaches may be used to reduce the costs of rework:**
    a. Change anticipation:
    b. Change tolerance
8. **Coping with change: So, two ways of coping with change and changing system requirements:**
    a. System prototyping
    b. Incremental delivery

Unit-1: Introduction – Q&A

9. **Process improvement: Two quite different approaches to process improvement and change are used:**
    a. The process maturity approach
    b. The agile approach

10. **Agile Software Development: All of the agile methods that have been proposed share a number of common characteristics:**
    a. Program specification, design, and implementation are interleaved
    b. The system is developed as a series of frequent versions or increments
    c. Stakeholders involved in version specification and evaluation
    d. Extensive tool support (e.g. automated testing tools) used to support development
    e. Minimal documentation - focus on working code
11. **The Agile Manifesto consists of four key values:**
    a. Individuals and interactions over processes and tools
    b. Working software over comprehensive documentation
    c. Customer collaboration over contract negotiation
    d. Responding to change over following a plan